

CS 4872 Replication Project: Tree of Thoughts (ToT)

Ethan Zhang, Vinny Huang, Veronica Starchenko, Phoebe Wang

Project Github: <https://github.com/empzhang22/cs4782-tree-of-thoughts>

Introduction

As autoregressive systems, standard LLMs generate text token by token, and cannot revisit or revise earlier decisions. This inability to recover from mistakes or explore alternative strategies limits performance on problems that require multi-step planning... cue Tree of Thoughts.

This project sought to reimplement the proposed prompting technique from the paper, *Tree of Thoughts: Deliberate Problem Solving with Large Language Models* by Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts (ToT), is a prompting method that reframes problem solving as a search over a tree of intermediate reasoning steps (“thoughts”). Rather than one linear chain, ToT (1) decomposes problems into thought steps, (2) generates multiple candidate thoughts at each step, (3) evaluates each using the LLM itself as a heuristic, and (4) searches the tree with BFS or DFS. This generalizes the popular Chain of Thought (CoT) prompting technique by enabling branching and backtracking. In the paper, ToT was shown to outperform CoT baselines on three different tasks: Game of 24, Creative Writing, and Mini Crosswords.

Chosen Result

We focused primarily on recreating **Table 2: Game of 24**, which compares the success rates of Input-Output (IO), Chain of Thought (CoT), CoT with Self-Consistency (CoT-SC), and Tree of Thoughts (ToT) prompting methods on 100 difficult programs from **4nums.com**. The game is producing 24 from four given numbers only using basic arithmetic operations. We chose this benchmark because it is intuitive, has a clear binary success criterion, and highlights reasoning performance effectively.

We also recreated **Table 3: Mini Crosswords**, where models solve 5x5 crossword puzzles using clue-based reasoning, and explored **Figure 5: Creative Writing**, where models generate coherent passages from four random sentences. Due to time and cost constraints, we evaluated creative writing outputs using LLM-based coherence judgments instead of human evaluators like Amazon Mechanical Turk. Across all tasks, we used the same datasets and methodology as the original paper.

The original paper, using the GPT-4 model, found that ToT significantly outperformed standard prompting methods. Even the best CoT baseline achieved only 9% success on Game of 24, while ToT methods performed substantially better, demonstrating the benefits of exploring and backtracking through multiple reasoning paths. Our goal was to reproduce these findings.

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	74%
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

Method	Success Rate (%)		
	Letter	Word	Game
IO	38.7	14	0
CoT	40.6	15.6	1
ToT (ours)	78	60	20
+best state	82.4	67.5	35
-prune	65.4	41.5	5
-backtrack	54.6	20	5

Table 3: Mini Crosswords results.

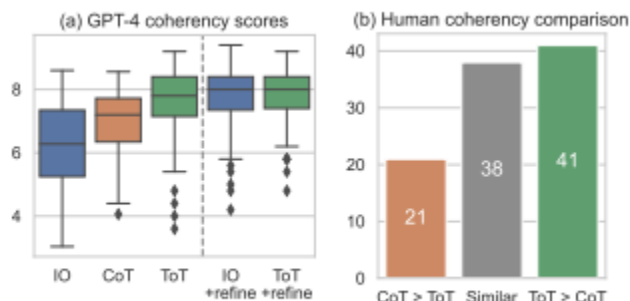


Figure 5: Creative Writing results.

Methodology

We originally used **DeepSeek-r1:8b** to test if ToT can boost the robustness of local models to the level of frontier models which could serve as a free-of-cost alternative, but we encountered many challenges pertaining to the model following instructions and generating meaningful intermediate reasoning steps.

As such, we pivoted to use **Gemini 2.5 Flash** via the Google AI API as our backbone LLM, replacing the GPT-4 used in the original paper. Gemini 2.5 Flash is a cost-efficient model suited to multi-call inference workflows like ToT, which requires multiple LLM calls per problem. The paper’s GPT-4 model was very costly, and the resource/token cost of ToT was too expensive, which is why we explored alternate models.

Our ToT-BFS implementation followed the paper's four stages. Each problem is decomposed into three thought steps, where each step selects two numbers, applies an operation, and records the remaining values. At each step, a **propose prompt** generates candidate next steps; a **value prompt** then rates each state as **sure**, **likely**, or **impossible**. Impossible states are pruned; the top-5 remaining states advance (beam width $b=5$, matching the paper). A solution is confirmed programmatically by verifying the output expression equals 24 using each input number exactly once. Prompts were taken directly from the official repository (princeton-nlp/tree-of-thought-llm).

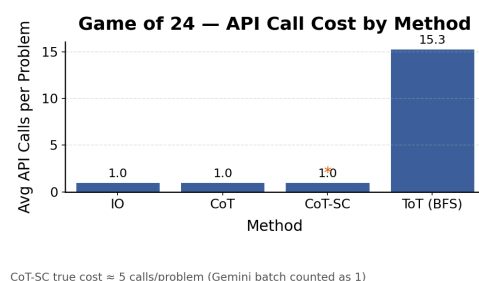
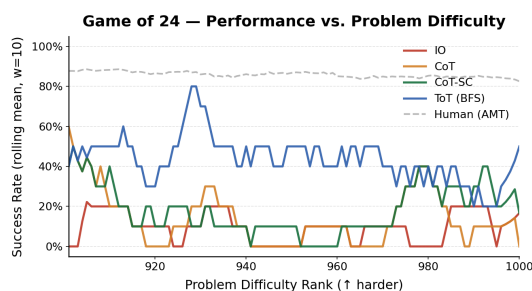
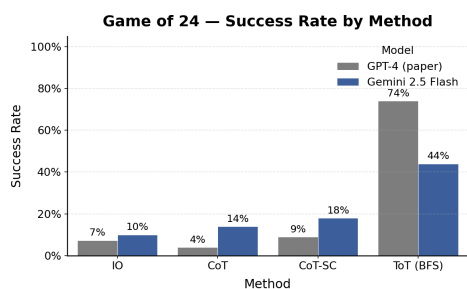
Results and Analysis

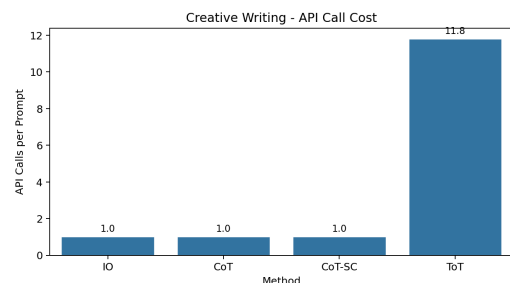
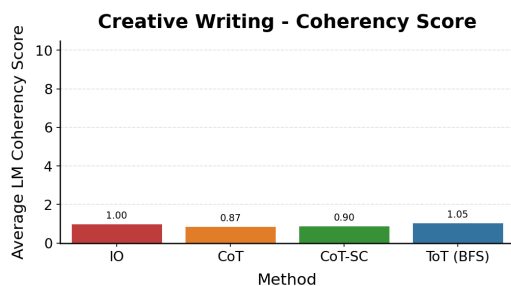
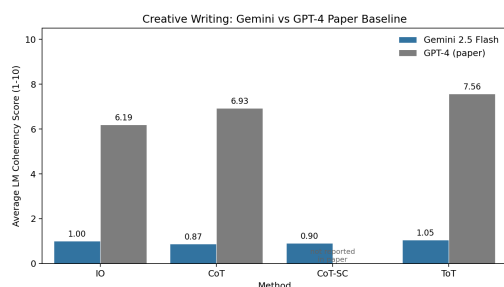
While our ToT implementation did not achieve similar levels of success against the original GPT-4 implementation, the relative ordering across methods is preserved: ToT is roughly $5\times$ better for the Game of 24 and $2\times$ better for the Crossword puzzles when compared with the CoT baselines. This confirms that ToT significantly improves a model’s reasoning capability in multi-step logic problems.

The primary source of the gap is model capability. GPT-4 is stronger at structured arithmetic reasoning; Gemini 2.5 Flash exhibited more frequent arithmetic errors and failures, as seen in the Performance vs. Problem Difficulty figure where all prompting methods were less successful than human performance (although ToT-BFS degrades more gracefully on the hardest puzzles, compared to IO and CoT). Similarly, no model achieved 100% accuracy across the crossword puzzles, and even the ToT implementation only achieved a 5.9% game accuracy.

One of the challenges we faced was that the creative writing prompts seemed largely unsuccessful in comparing the strength of ToT over CoT. All the prompting methodologies achieved very similar coherency means at around 1, although ToT had a single outlier as a 9 out of 10. It is unclear whether the low performances are a result of some weakness of the model in long-term story planning or in evaluating stories for “coherency”. In addition, we identify prompt sensitivity was also a challenge: minor formatting differences led to malformed outputs requiring additional parsing logic.

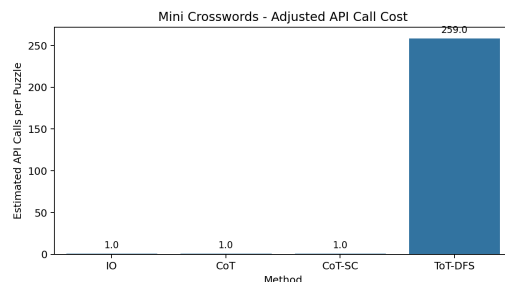
A significant tradeoff is resource cost. For instance: with the 24 game, each branch requires multiple LLM calls to generate, execute, and evaluate each “node” in the tree; token usage increases exponentially. For example, if we use $b=5$ BFS over 3 steps, each problem requires 15+ LLM calls, totaling over 1,500 calls across the benchmark. Similarly, exploring a myriad of crossword solutions involved heavy branching, leading to individual games taking several minutes to solve over 200+ LLM calls.



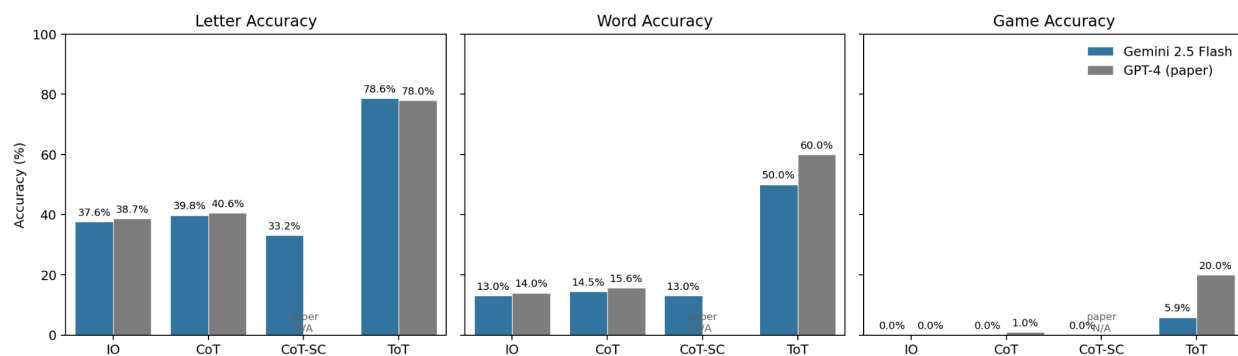


Mini Crosswords Results (Gemini 2.5 Flash)

Method	N	Letter Accuracy	Word Accuracy	Game Accuracy
IO	20	37.6%	13.0%	0.0%
CoT	20	39.8%	14.5%	0.0%
CoT-SC	20	33.2%	13.0%	0.0%
ToT-DFS	17	78.6%	50.0%	5.9%



Mini Crosswords: Gemini vs GPT-4 Paper Baseline



Reflection

Overall, we successfully reproduced the core finding of Yao et al. (2023): deliberate tree search over LLM-generated thoughts outperforms chain-of-thought prompting on the Game of 24 in lighter models.

A lesson learned is that the fundamental model is a limiting factor that can't be overcome by prompting. While our experiment implied that the ToT advantage is robust across models — the trend holds on Gemini 2.5 Flash despite an absolute gap from GPT-4 — discussion with other groups revealed that newer models (GPT 5.5) are so strong that ToT doesn't significantly improve responses. On the other end of the spectrum, low-parameter, local models like DeepSeek-r1:8b don't produce strong enough reasoning steps to meaningfully benefit from ToT, demonstrating that **the original paper has the hidden assumption that there is a baseline level of model intelligence required for ToT to function.**

We also learned that cost and computation are significant constraints and any practical deployment of ToT must weigh these costs against performance gains. As the ToT becomes more advanced (more branches, more depth, etc), it performs significantly better, but it also consumes more tokens and takes much longer. Some complex problems can take several minutes to run. ToT is not ideal for simple questions and its latency costs are likely why it isn't more popular despite strong performance.

Future steps we are curious about include using mixed models, like a cheaper model for proposal steps and a stronger model for evaluation to improve cost efficiency but maintain strong reasoning. Also, labeling failure cases to determine what caused errors.

References

1. **Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023).** *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. Advances in Neural Information Processing Systems (NeurIPS 2023). arXiv:2305.10601.
2. **Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., & Zhou, D. (2022).** Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. <https://arxiv.org/abs/2201.11903>
3. **Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2022).** Self-Consistency Improves Chain of Thought Reasoning in Language Models. <https://arxiv.org/abs/2203.11171>
4. **Princeton NLP Group.** *tree-of-thought-llm: Official repository for Tree of Thoughts*. GitHub. <https://github.com/princeton-nlp/tree-of-thought-llm>
5. **Google DeepMind.** *Gemini 2.5 Flash*. Google AI API. <https://ai.google.dev/>
6. **4nums.com.** *Game of 24 Dataset*. <https://www.4nums.com/game/difficulties/>
7. **OpenAI.** *GPT-4 Technical Report*. (2023). arXiv:2303.08774.